

CPT102-2526 Exercise 13 Hashing - Soln

In Lecture 13, we have learned about separate chaining, resizing hash tables, and good hash codes. You will use the techniques you have learned to solve the problems in this exercise.

Problem 13.1 Hashing Collision

Find a large set of strings, each of the same length, that have the same `hashCode()` value, using the `hashCode()` implementation for `String` as specified in Lecture 13, shown here again:

```
public int hashCode() {  
    int hash = 0;  
    for (int i = 0; i < length(); i++)  
        hash = (hash * 31) + charAt(i);  
    return hash;  
}
```

Hint: Use the fact that "Aa" and "BB" hash to the same `hashCode()` value which is 2112.

Solution:

Following the hint, any string of length $2N$ that is formed by concatenating these two strings together in any order (e.g., BBBBBB, AaAaAa, BBaAaBB, AaBBBBB) will hash to the same value. Use such patterns to generate the strings.

Problem 13.2 Hashing and Searching

Suppose we wish to repeatedly search a linked list of length N elements, each of which contains a very long string key. How might we take advantage of the hash value when searching the list for an element with a given key?

Hint: Use hashing to avoid comparing the very long string key.

Solution:

Precompute the hash value of each string in the list. When searching for a key t , compare its hash value to the hash value of a string s . Only compare the string s and t if their hash values are equal.

Problem 13.3 Deletion

Write pseudocode for deletion in a separate chaining hash table.

Hint: Similar to insert, search first.

Solution:

delete(key):

 compute hash $h = \text{hashCode}(\text{key})$

 remove key from the bucket h if key exists in the linked list

This is the end of CPT102-2526 Exercise 13.